

# AuRAG

Industrial Knowledge Intelligence and Unified Operations Agent

## Document purpose

A detailed engineering reference for the current repository implementation: architecture, data flow, retrieval, agent behavior, APIs, operator workflows, deployment, testing, and known boundaries.

|                     |                                             |
|---------------------|---------------------------------------------|
| Repository snapshot | July 22, 2026                               |
| Primary audience    | Engineers, architects, operators, reviewers |
| Runtime shape       | Next.js + FastAPI + Neo4j + Qdrant + Redis  |
| Document status     | Implementation-aligned reference            |

Prepared from source code, configuration, tests, deployment notes, and project documentation.

|           |                                                                                      |
|-----------|--------------------------------------------------------------------------------------|
| <b>01</b> | Executive Summary<br>What AuRAG does and where its value comes from                  |
| <b>02</b> | System Architecture<br>Runtime topology and architectural boundaries                 |
| <b>03</b> | Ingestion and Knowledge Graph<br>How plant documents become linked evidence          |
| <b>04</b> | Retrieval and Agent Orchestration<br>Hybrid retrieval, routing, and grounded answers |
| <b>05</b> | Operator Experience<br>Frontend workspaces and action loops                          |
| <b>06</b> | API and Domain Contracts<br>Endpoints, payloads, and state transitions               |
| <b>07</b> | Evaluation and Observability<br>RAGAS, durable records, health, and events           |
| <b>08</b> | Deployment and Operations<br>Local stack, managed services, and release checks       |
| <b>09</b> | Security, Reliability, and Data Governance<br>Failure modes and trust boundaries     |
| <b>10</b> | Testing and Validation<br>Automated coverage and recorded evidence                   |
| <b>11</b> | Limitations and Roadmap<br>Current scope boundaries and next closure items           |
| <b>12</b> | Appendix<br>Repository map, configuration, and endpoint index                        |

## Executive Summary

AuRAG is an industrial knowledge intelligence platform that turns fragmented plant records, equipment history, procedures, regulatory clauses, and telemetry signals into a grounded operational workflow. The primary interaction is a single operator console: ask a question, inspect the evidence graph, see the routed specialist, and move from a detected signal to a reviewed work order.

The implementation combines a graph-first data model with multiple retrieval strategies. Neo4j stores the operational relationships and is also used for native vector search. Qdrant supplies an independent dense-vector index, BM25 supplies keyword recall, graph traversal supplies entity-aware evidence, and a local cross-encoder reranks the fused candidates.

The application is intentionally fail-open at several boundaries. A scoring failure does not remove an answer, a mem0 failure does not block chat, a provider failure is surfaced as a clean API error, and work-order decisions use optimistic version checks to prevent stale edits from silently overwriting operator changes.

### Core product proof

Natural-language question -> specialist routing -> grounded answer -> citations -> graph evidence -> asynchronous quality score.

| Capability        | Current implementation                                                                                                                        |
|-------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| Unified interface | Next.js operator console with Command Center, Investigation, Predictive Watch, Evaluation, Knowledge Risk, Comparison, and Work Orders views. |
| Reasoning         | LangGraph Supervisor routes to Copilot, RCA, Compliance, or Lessons Learned.                                                                  |
| Grounding         | Citation-whitelisted LLM responses with retrieved context and typed graph paths.                                                              |
| Actionability     | Telemetry warnings create durable predictive events and editable, accept/reject work-order drafts.                                            |
| Quality           | Async RAGAS scoring for faithfulness, context precision, and answer relevancy, persisted as EvaluationRun.                                    |

## System Architecture

The system is organized into five runtime planes: user experience, application API, knowledge intelligence, persistence/runtime services, and model providers. The FastAPI layer is an adapter and orchestration boundary; domain behavior remains in agents, retrieval, telemetry, ingestion, evaluation, and service modules.

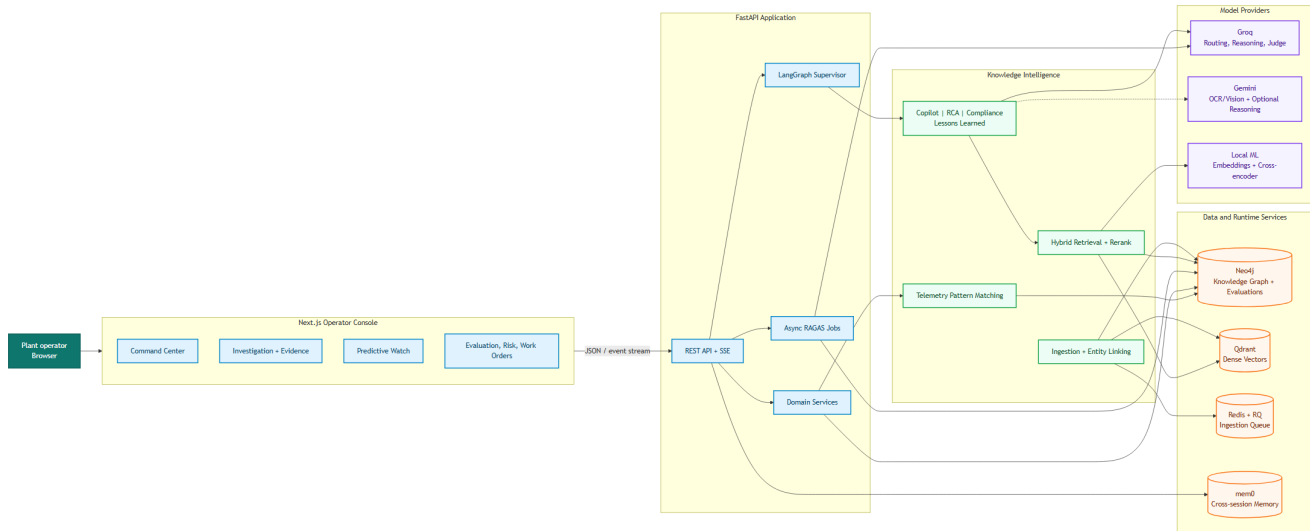


Figure 1. Current AuRAG platform architecture.

The browser never connects directly to Neo4j. Graph evidence is requested through `POST /api/graph`, which resolves answer-specific typed paths server-side. This keeps graph credentials and query construction on the backend and makes the frontend contract stable.

| Layer        | Responsibilities                                                                                                   | Key modules                             |
|--------------|--------------------------------------------------------------------------------------------------------------------|-----------------------------------------|
| Frontend     | Operator navigation, chat, graph canvas, telemetry controls, evaluations, risk, comparison, and work-order review. | frontend/app, frontend/components       |
| API          | Request validation, dependency injection, error flattening, REST and SSE contracts.                                | backend/app/api, backend/app/main.py    |
| Intelligence | Intent classification, specialized reasoning, retrieval fusion, telemetry matching, ingestion.                     | agents, retrieval, telemetry, ingestion |
| Persistence  | Graph entities, vectors, queue state, memory, evaluations, predictive events, decision audit.                      | Neo4j, Qdrant, Redis/RQ, mem0           |
| Providers    | Routing/reasoning/judging, OCR/vision, embeddings, reranking.                                                      | Groq, Gemini, sentence-transformers     |

## Ingestion and Knowledge Graph

All ingestion paths converge on `ingestion.pipeline.ingest_file()`. Clean text is parsed directly, scanned images use Tesseract with a Gemini fallback, and P&ID; diagrams use Gemini vision to produce page summaries, equipment mentions, and connection candidates. The pipeline uses closed-world matching: only known equipment and personnel are linked, preventing uncontrolled graph expansion from model hallucinations.

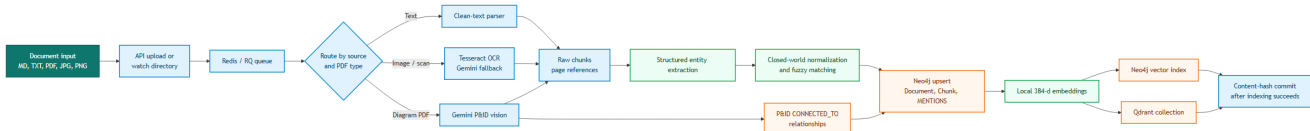


Figure 2. Ingestion convergence and dual vector indexing.

Document identity is content-hash based. Unchanged files are fast no-ops. Changed documents clear and rebuild their chunks and P&ID; relationships, then update the content hash only after Neo4j and Qdrant indexing succeeds. That ordering preserves retryability when vector indexing or a provider call fails.

Neo4j is the relationship authority. The current schema includes unique constraints for equipment, documents, chunks, people, work orders, failure events, regulatory clauses, procedures, evaluations, predictive events, notifications, and work-order decisions. A 384-dimensional cosine vector index is created over Chunk embeddings.

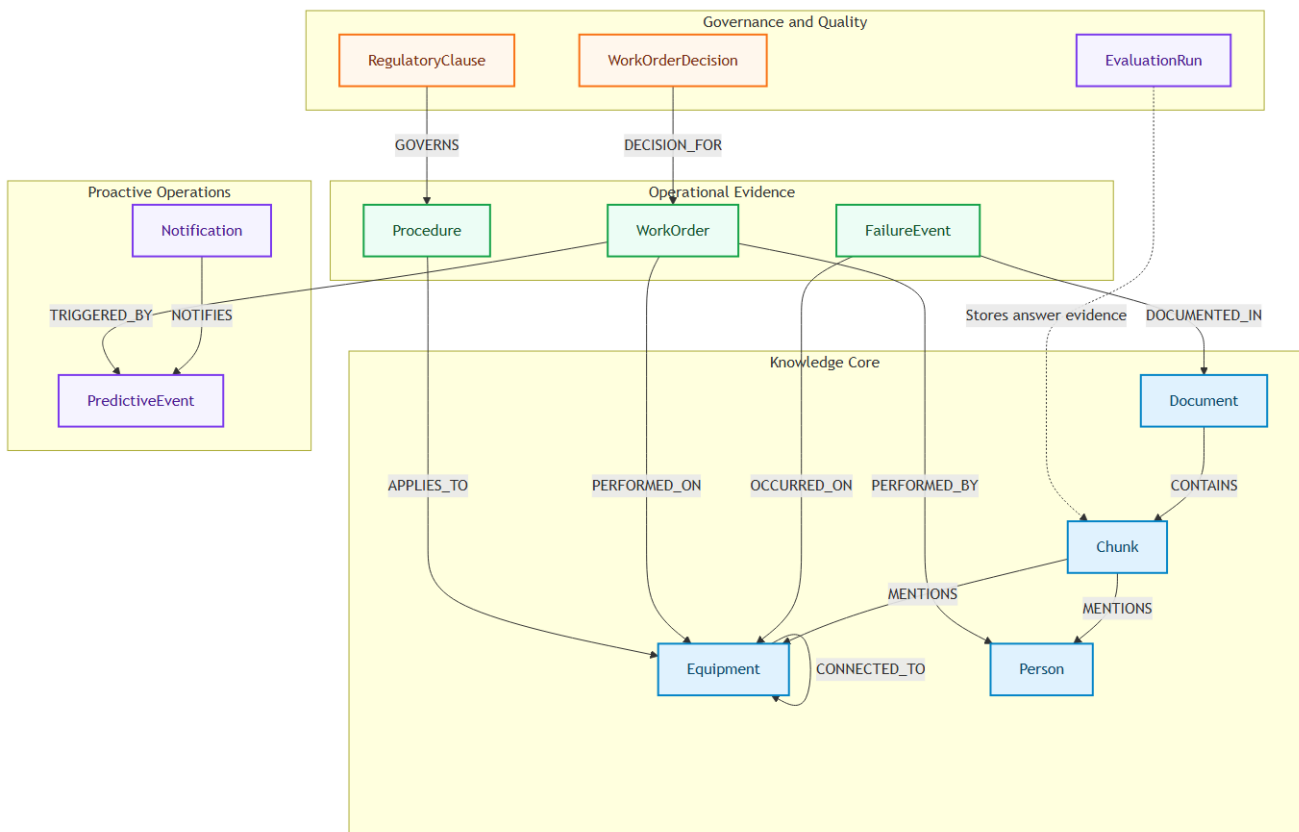


Figure 3. Persisted knowledge and operational relationship model.

## Retrieval and Agent Orchestration

The Supervisor is a LangGraph StateGraph. It classifies each query into one of four intents, applies a confidence floor of 0.6, and routes low-confidence or unknown classifications to Copilot as the general fallback. The classifier's raw intent and the agent that actually ran are both retained for debugging and observability.

| Route           | Question shape                                       | Evidence behavior                                                             |
|-----------------|------------------------------------------------------|-------------------------------------------------------------------------------|
| Copilot         | General plant Q&A; and document lookup.              | Hybrid GraphRAG: vectors, Qdrant, BM25, graph traversal, filters, and rerank. |
| RCA             | Why did a specific asset fail?                       | Entity-anchored failure, work-order, procedure, and document traversal.       |
| Compliance      | Is an asset compliant or what does a clause require? | Clause-first evidence shaping, gap detection, quotations, and citations.      |
| Lessons Learned | What patterns recur across incidents?                | Cross-incident scan and pattern synthesis over failure/work-order history.    |

Hybrid retrieval collects up to ten candidates per source, deduplicates by source key, applies explicit equipment/name and year filters, then reranks the remaining passages with the configured local cross-encoder. The answer layer receives a context-key whitelist, and model citations are filtered against that whitelist before they can reach the UI or graph resolver.

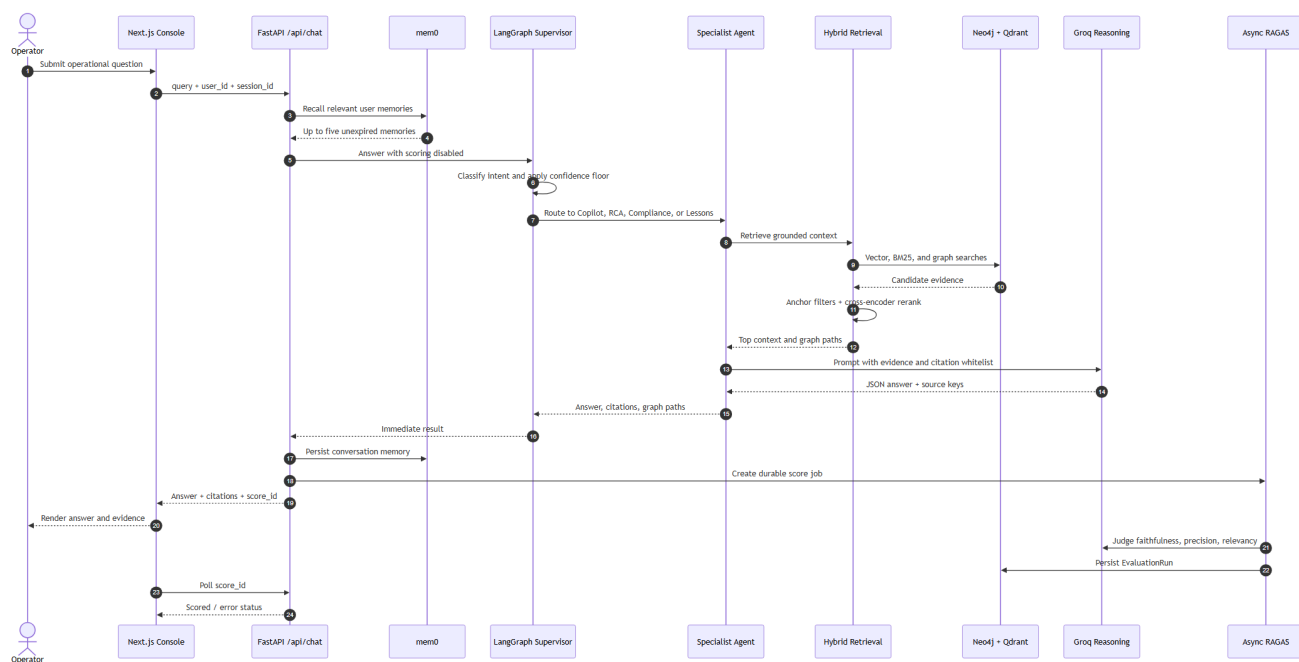


Figure 4. Query lifecycle from operator question to async score.

### Why the graph matters

Dense retrieval finds semantically similar passages. Graph traversal adds the relationship path that connects a failure, asset, work order, procedure, person, clause, or predictive event into an inspectable answer trail.

## Operator Experience

The frontend is a Next.js 16 application with a persistent shell, route-aware titles, readiness polling, responsive workspaces, and typed API wrappers. The UI is structured around operational decisions rather than generic chat alone.

| Workspace        | Operator task                         | Key behavior                                                                                                    |
|------------------|---------------------------------------|-----------------------------------------------------------------------------------------------------------------|
| Command Center   | Start from a unified plant overview.  | Coverage tiles, workflow entry points, platform readiness, and guided operational prompts.                      |
| Investigation    | Ask, inspect, and validate an answer. | Chat panel, routed-agent badge, citations, RAGAS pills, graph canvas, and evidence details.                     |
| Predictive Watch | Move from signal to intervention.     | Equipment selector, drift slider, live scan, warning, durable event, draft work order, edit, accept, or reject. |
| Knowledge Risk   | Protect retiring expertise.           | Risk score, asset coverage, uncovered equipment, evidence, and recommended capture/successor actions.           |
| Comparison       | Make GraphRAG value visible.          | Side-by-side graph answer and dense-only baseline with source overlap and relationship evidence.                |
| Evaluation       | Review quality over time.             | Summary metrics, trend chart, filters, paginated EvaluationRun history, low-faithfulness queue.                 |
| Work Orders      | Review action state safely.           | Draft/In Review/Approved/Rejected status, versioned edits, decision reasons, and audit history.                 |

### Operator trust loop

Every answer exposes what ran, what sources were used, how the answer maps into the graph, and whether asynchronous quality scoring completed. Predictive actions remain reviewable and versioned rather than silently becoming plant work.

## API and Domain Contracts

The FastAPI application mounts routers under `/api`. Endpoint handlers validate input, call domain services, and translate failures into stable JSON error shapes. The current CORS implementation reads `BACKEND_CORS_ORIGINS` from the environment.

| Surface                                  | Method             | Purpose                                                                                  |
|------------------------------------------|--------------------|------------------------------------------------------------------------------------------|
| <code>/health/live</code>                | GET                | Process liveness; does not require all providers.                                        |
| <code>/health/ready</code>               | GET                | Dependency readiness across Neo4j, Qdrant, Redis, Groq, Gemini, and mem0.                |
| <code>/chat</code>                       | POST               | Recall memory, route query, return answer/citations/graph paths, create async score job. |
| <code>/chat/scores/{score_id}</code>     | GET                | Poll in-memory or durable EvaluationRun score state.                                     |
| <code>/graph</code>                      | POST               | Resolve typed answer graph paths into nodes and relationships.                           |
| <code>/telemetry/scan</code>             | POST               | Generate synthetic reading, match failure signature, and return warning.                 |
| <code>/telemetry/draft</code>            | POST               | Create or retrieve a durable predictive work-order draft.                                |
| <code>/events/predictive</code>          | GET                | Server-Sent Events stream with keep-alives and cursor support.                           |
| <code>/notifications</code>              | GET/POST           | List predictive notifications and mark them read.                                        |
| <code>/work-orders</code>                | GET/POST/<br>PATCH | List, create, and edit persisted work-order drafts.                                      |
| <code>/work-orders/{id}/decisions</code> | POST               | Accept or reject with version check and reason for reject.                               |
| <code>/knowledge-risk</code>             | GET                | Rank personnel knowledge-retirement risk and asset coverage.                             |
| <code>/comparison</code>                 | POST               | Run GraphRAG versus dense-only retrieval comparison.                                     |
| <code>/evaluations</code>                | GET                | Query evaluation history and aggregate summary.                                          |
| <code>/ingestion/documents</code>        | POST               | Accept a document upload and enqueue ingestion.                                          |

Work-order edits and decisions use optimistic concurrency. The client sends `expected_version`; a mismatch produces a conflict rather than an overwrite. Every edit and decision becomes a `WorkOrderDecision` record with actor, reason, timestamps, and version transition.

```
POST /api/work-orders/{id}/decisions
{
  "decision": "reject",
  "expected_version": 2,
  "reason": "Field inspection found no actionable anomaly",
  "actor": "local-operator"
}
```

## Evaluation and Observability

Chat returns the answer first. When retrieved context exists, the API creates a durable EvaluationRun and starts a daemon-thread scoring job. The frontend polls the score endpoint and updates the answer in place. This keeps operator latency independent from judge-model latency while preserving a quality trail.

| Metric                      | Meaning                                                 | Product use                                         |
|-----------------------------|---------------------------------------------------------|-----------------------------------------------------|
| Faithfulness                | Whether the answer is supported by retrieved context.   | Low-faithfulness warning and review queue.          |
| Context precision           | Whether retrieved context is useful and focused.        | Retrieval quality signal.                           |
| Answer relevancy            | Whether the response addresses the question.            | Answer usefulness signal.                           |
| Routing confidence          | Classifier confidence before confidence-floor fallback. | Explains the selected specialist and misroute risk. |
| Graph relationship evidence | Count of answer-specific graph paths.                   | Makes relational grounding visible.                 |

### Fail-open scoring

If RAGAS dependencies, judge calls, or durable score updates fail, the answer remains available. The UI distinguishes scoring, scored, skipped-no-context, delayed, and error states.

Predictive operations use two complementary delivery modes. The interactive UI scans synthetic drift and persists a PredictiveEvent when the warning threshold fires. The background TelemetryWorker scans configured equipment on an interval, deduplicates events through a cooldown window, and publishes new notification IDs for the SSE stream.

```
TelemetryWorker.run_cycle(session)
drift = scheduled test phase
for equipment in configured_tags:
    reading = generate_reading(...)
    warning = match_reading(...)
    if warning:
        event = create_predictive_event(..., cooldown_seconds=1800)
        publish_event(event.id)
```

## Deployment and Operations

The repository supports a persistent local stack with Docker Compose: Neo4j 5.26 Community, Redis 7 Alpine, and Qdrant 1.18.0. Named volumes preserve graph, queue, and vector data across container restarts. The API image uses Python 3.12, runs as an unprivileged user, includes Tesseract, and exposes a process-only liveness check.

| Component         | Local command / location                                              | Operational note                                         |
|-------------------|-----------------------------------------------------------------------|----------------------------------------------------------|
| Infrastructure    | <code>docker compose -f infra/docker-compose.yml up -d</code>         | Neo4j, Redis, and Qdrant use named volumes.              |
| API               | <code>.venv\Scripts\python.exe -m uvicorn backend.app.main:app</code> | Run from repository root; port 8000.                     |
| Frontend          | <code>npm.cmd --prefix frontend run dev</code>                        | Next.js dev server; usually port 3000.                   |
| Telemetry         | <code>python -m telemetry.worker</code>                               | Continuous synthetic scan and event publication.         |
| Ingestion watcher | <code>python -m ingestion.workers.watcher</code>                      | Watch directory to Redis/RQ handoff.                     |
| RQ worker         | <code>rq worker aurag-ingest</code>                                   | Use SimpleWorker on Windows.                             |
| Managed API       | <code>render.yaml</code>                                              | Render API plus telemetry and ingestion worker services. |
| Managed UI        | <code>frontend/vercel.json</code>                                     | Vercel project rooted at frontend/.                      |

Bootstrap is deliberately explicit: install dependencies, start infrastructure, apply schema and seed, then rebuild both vector indexes. Readiness is stricter than liveness and requires configured external providers to respond.

```
.\scripts\bootstrap.ps1
.\scripts\bootstrap.ps1 -SkipInstall -SkipInfra -InitializeData
.\scripts\smoke.ps1
```

## Security, Reliability, and Data Governance

AuRAG is currently a local/demo-oriented operator application rather than a multi-tenant security product. There is no authentication or authorization layer in the repository. User identity is used for mem0 scoping and work-order actor attribution, but it is not an access-control boundary.

| Concern          | Current control                                                 | Residual risk / next action                                          |
|------------------|-----------------------------------------------------------------|----------------------------------------------------------------------|
| Secrets          | Provider keys come from .env; Docker image excludes local .env. | Use managed secret storage and rotation for production.              |
| Graph access     | Browser accesses graph through backend proxy only.              | Add auth and query-level authorization before multi-tenant use.      |
| LLM citations    | Citations are whitelist-filtered to retrieved context keys.     | Add stronger claim-level verification for high-stakes workflows.     |
| Provider failure | API endpoints return structured 503 errors; scoring fails open. | Add circuit breakers, retry budgets, and operator diagnostics.       |
| Stale edits      | Work orders use expected_version and audit decisions.           | Add broader idempotency keys to all mutating external calls.         |
| Memory           | mem0 records are user-scoped and expire after 30 days.          | Document retention policy and provide deletion controls.             |
| Uploads          | 50 MB default cap, storage backend abstraction, async queue.    | Add malware scanning, content-type policy, and tenant isolation.     |
| CORS             | Configured origins are read from BACKEND_CORS_ORIGINS.          | Verify exact Vercel production and preview origins in browser smoke. |

### Important deployment boundary

Readiness requires Neo4j, Qdrant, Redis, Groq, Gemini, and configured mem0. A service can be alive while the operator workflow is not ready; release checks must use both endpoints.

## Testing and Validation

The repository contains 107 Python test functions across agent, backend, evaluation, ingestion, retrieval, and telemetry suites, plus frontend Vitest and Playwright coverage. The tests are organized by domain rather than by endpoint alone, which keeps pure logic independently testable and API adapters thin.

| Area       | Python test files                     | Primary contract tested                                                                               |
|------------|---------------------------------------|-------------------------------------------------------------------------------------------------------|
| Agents     | tests/agents/*                        | Routing, citations, LLM wrappers, RCA, compliance, lessons learned, validation.                       |
| Backend    | tests/backend/*                       | Health, chat, graph, telemetry, events, evaluation, comparison, memory, work orders, risk, ingestion. |
| Ingestion  | tests/ingestion/*                     | Retry, storage, OCR fallback, extraction validation, worker behavior, Gemini utility.                 |
| Retrieval  | tests/retrieval/*                     | Plain vector, hybrid fusion, rerank, Qdrant configuration.                                            |
| Telemetry  | tests/telemetry/*                     | Worker cycle behavior and precision thresholds.                                                       |
| Evaluation | tests/evaluation/*                    | RAGAS validation and comparison benchmark.                                                            |
| Frontend   | frontend/**/*test.tsx + e2e/*.spec.ts | Component contracts, mobile workflows, operator workflows, API/session utilities.                     |

Recorded project evidence includes ingestion ground truth of 13/13 expected mentions with zero spurious mentions, hybrid retrieval ground truth of 5/5 queries, agent citation validation of 8/8, supervisor validation of 8/8, monotonic telemetry drift sweeps across six seeded events, and passing frontend lint/build evidence. These are repository-reported validation results; run the current smoke suite against live dependencies before calling a deployment ready.

### Verification command set

Use the full smoke script for live dependency validation. Use `-SkipMutatingChecks` when only read-oriented checks are appropriate; the full script intentionally leaves a generated smoke work order in Rejected status.

```

.\.venv\Scripts\python.exe -m pytest
npm.cmd --prefix frontend test
npm.cmd --prefix frontend run lint
npm.cmd --prefix frontend run build
.\scripts\smoke.ps1

```

## Limitations and Roadmap

The current repository has moved beyond the earlier July 20 closure snapshot: knowledge-retirement risk, GraphRAG comparison, durable predictive events, work-order decisions, evaluation history, configurable CORS, and mem0 adapters are now present in code. The following boundaries remain relevant to engineering decisions.

| Boundary              | Current state                                                                           | Recommended closure                                                                  |
|-----------------------|-----------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| Authentication        | No auth/permissions; local operator identity only.                                      | Introduce identity provider, role model, tenant boundaries, and audit access policy. |
| Telemetry realism     | Synthetic readings and one seeded failure mode per equipment tag.                       | Add real historian integration and multiple competing failure signatures per asset.  |
| P&ID; semantics       | Connections are linked when endpoints match known equipment.                            | Add line/instrument entities and validate connectivity semantics at scale.           |
| RAGAS baseline        | Durable history and trend endpoints exist, but judge scores remain model/run sensitive. | Version evaluation prompts/models and maintain stable benchmark baselines.           |
| Memory operations     | Fail-open mem0 adapter with TTL metadata.                                               | Add retention controls, deletion workflow, and provider health metrics.              |
| Deployment acceptance | Render/Vercel artifacts and smoke script exist.                                         | Run fresh managed browser smoke including exact production CORS.                     |
| Release governance    | Repository snapshot is rich but release status depends on live infrastructure.          | Commit a reproducible release baseline and attach smoke evidence.                    |

The practical next milestone is a truthful end-to-end environment: provision or restore Neo4j, rebuild the Python runtime if needed, start Qdrant and Redis, apply seed and vector indexes, start API and frontend, and run the smoke suite. Only after that should a managed deployment be labeled operationally ready.

## Repository map

| Path        | Role                                                                          |
|-------------|-------------------------------------------------------------------------------|
| backend/    | FastAPI application, API routers, core adapters, and domain services.         |
| agents/     | Supervisor state graph, specialist agents, LLM wrappers, validation.          |
| retrieval/  | Embeddings, Neo4j vector, Qdrant, BM25, graph traversal, fusion, reranking.   |
| ingestion/  | Parsing, OCR, P&ID; vision, entity extraction, storage, watcher, queue tasks. |
| telemetry/  | Synthetic readings, signature matching, worker, warning and draft logic.      |
| evaluation/ | RAGAS scoring and validation.                                                 |
| frontend/   | Next.js operator console, typed API wrappers, unit/e2e tests.                 |
| infra/      | Docker Compose and Neo4j schema/seed.                                         |
| scripts/    | Bootstrap and live smoke verification.                                        |
| data/       | Synthetic plant source documents and scans.                                   |

## Key environment variables

| Variable                                              | Purpose                                          |
|-------------------------------------------------------|--------------------------------------------------|
| NEO4J_URI / NEO4J_USERNAME / NEO4J_PASSWORD           | Graph database connection.                       |
| QDRANT_URL / QDRANT_API_KEY                           | Dense vector store connection.                   |
| REDIS_URL                                             | Ingestion queue and worker connection.           |
| GROQ_API_KEY / GROQ_JUDGE_MODEL                       | Routing, reasoning, and judge models.            |
| GEMINI_API_KEY / GEMINI_INGESTION_MODEL               | OCR, P&ID; vision, and optional reasoning.       |
| MEM0_API_KEY / MEM0_DIR                               | Cross-session memory provider and runtime state. |
| BACKEND_CORS_ORIGINS                                  | Comma-separated allowed browser origins.         |
| INGEST_WATCH_DIR / INGEST_STORAGE_BACKEND             | Continuous ingestion source and storage mode.    |
| TELEMETRY_INTERVAL_SECONDS / TELEMETRY_EQUIPMENT_TAGS | Background predictive scan cadence and scope.    |

## Document provenance

This PDF was generated from the AuRAG repository at C:\ace\products\AuRAG on July 22, 2026. It reflects source code and project documentation available in that workspace. Runtime availability, provider credentials, database contents, and deployment readiness can change independently of the document and must be verified with the live smoke suite.

### End state

Core product implementation is broad and internally coherent. Production readiness remains an operational verification problem: infrastructure, credentials, browser CORS, and release evidence must all be green together.